



江南大学
JIANGNAN UNIVERSITY



人工智能与计算机学院
School of Artificial Intelligence and Computer Science

智能计算系统

Lab6: 模型推理优化



Lab6：模型推理优化

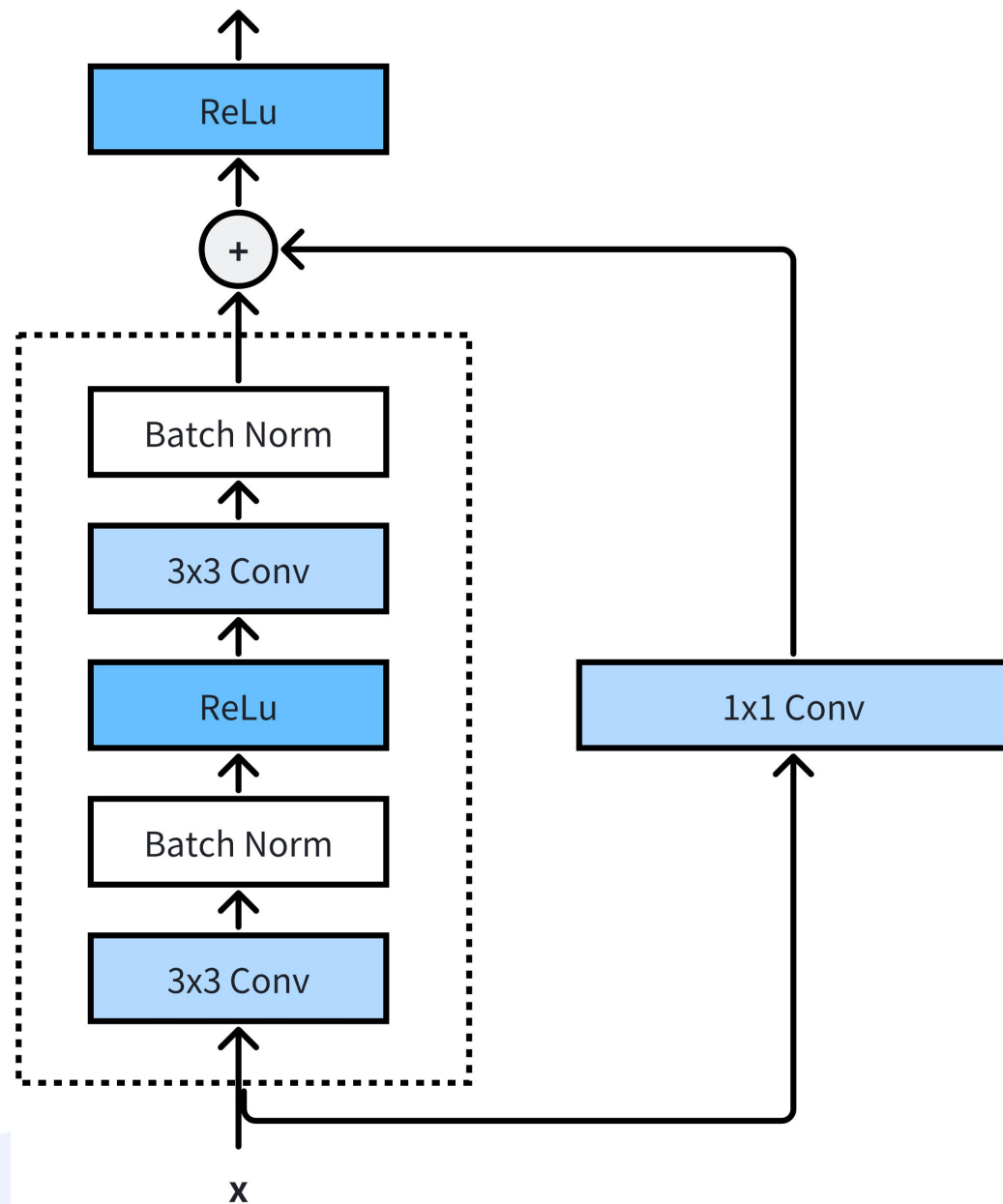
- ❑ 卷积神经网络（CNN）已广泛应用于图像分类、目标检测、医学影像分析和自动驾驶等场景。其参数共享机制虽可大幅度降低参数量，但核心卷积算子（Conv2d）的计算效率仍首先受限于框架默认实现的硬件适配性。
- ❑ 本实验以CNN经典网络 Resnet-18 为载体，旨在探索卷积算子的**全流程优化策略**。具体而言实验将应用等价算法以适应硬件计算单元特性，同时引入**量化技术**压缩数据位宽，在保持框架兼容性的前提下实现算子级加速。
- ❑ 实验结果表明，该优化可提升卷积操作的计算效率、降低内存占用，有利于在资源受限场景下的模型部署。

背景知识：ResNet网络

- Resnet网络由微软研究院的何凯明等人于2015年提出，其最大的创新在于引入了残差块（Residual Block），有效地解决了深度神经网络训练中的梯度消失和表示瓶颈问题。
- 残差块的核心思想是通过引入**跨层链接**，将输入直接传递到输出，从而形成一种“残差学习”的机制。这种设计允许反向传播的梯度直接流过整个网络，缓解了深度网络训练中的梯度消失问题。
- 因为ResNet的出现，让深度学习从而变成了可能，真正的实现了深度的概念。

背景知识：残差块设计

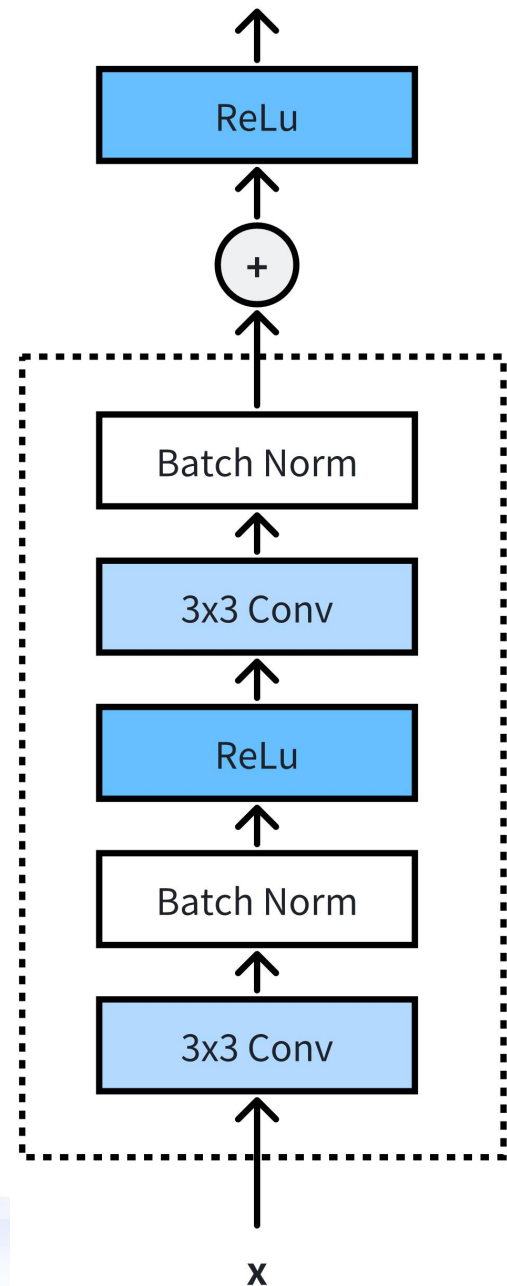
- ❑ 每个残差块包含主路径和跨层链接。
- ❑ 其中主路径通过两层卷积完成核心特征变换。
- ❑ 跨层链接通过 1×1 卷积调整通道数确保输入可直接参与相加。



背景知识：残差块设计-主路径

- ❑ 主路径中第一层卷积负责调整通道数以及下采样，后接批归一化和ReLU激活。
- ❑ 第二层卷积保持通道数不变，仅通过批归一化处理。
- ❑ 主路径对应pytorch代码如下：

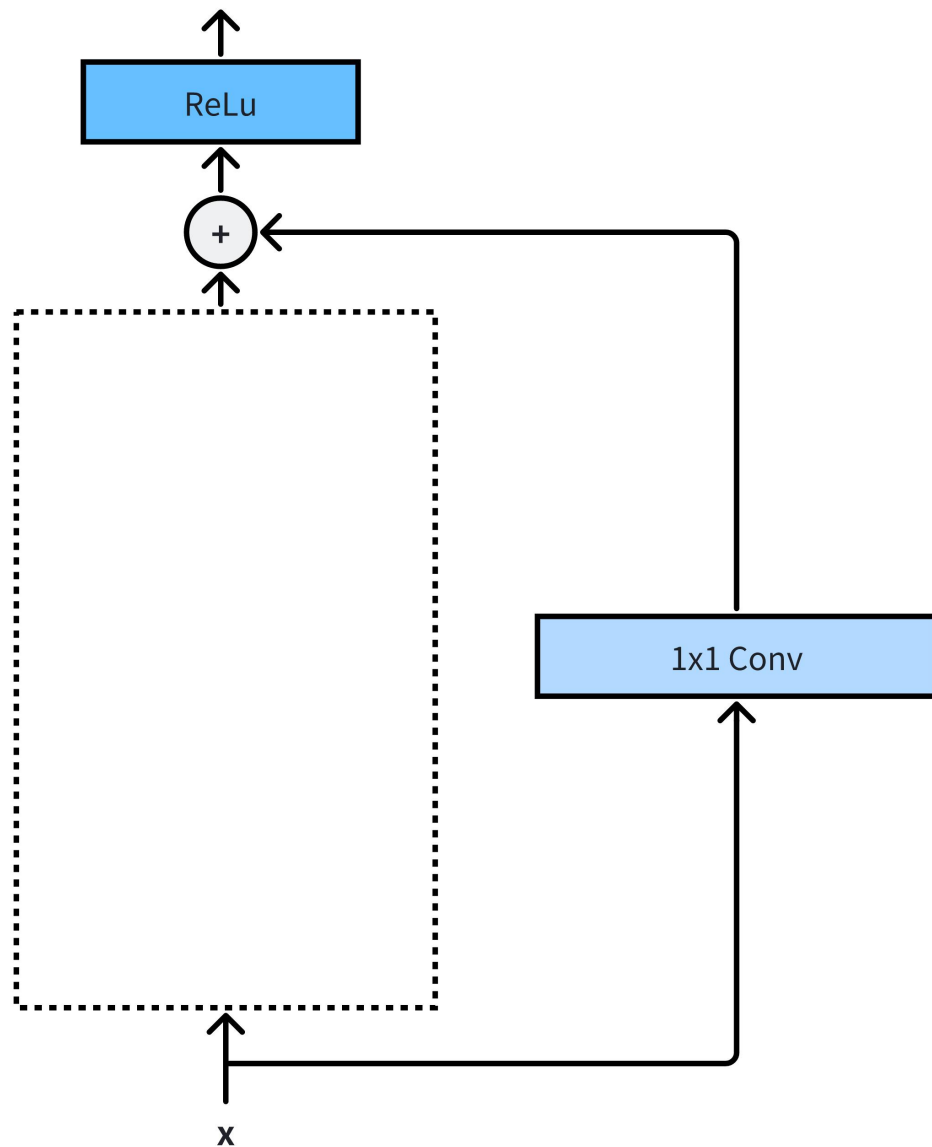
```
self.left = nn.Sequential(  
    nn.Conv2d(inchannel, outchannel, kernel_size=3, stride=stride,  
padding=1, bias=False),  
    nn.BatchNorm2d(outchannel),  
    nn.ReLU(inplace=True),  
    nn.Conv2d(outchannel, outchannel, kernel_size=3, stride=1,  
padding=1, bias=False),  
    nn.BatchNorm2d(outchannel)  
)
```



背景知识：残差块设计-跨层连接

- ❑ 跨层链接负责将输入直接传递到输出，确保梯度能有效回传。注意到仅当 $\text{stride} \neq 1$ (需要下采样) 或 $\text{inchannel} \neq \text{outchannel}$ (通道数不匹配) 时才启用 1×1 卷积调整维度。
- ❑ 1×1 卷积对应 $\text{kernel_size} = 1$, 仅调整通道数，不进行空间特征提取。
- ❑ 跨层链接对应pytorch代码如下：

```
self.shortcut = nn.Sequential(  
    Conv2d(inchannel, outchannel, kernel_size=1,  
    stride=stride, bias=False),  
    nn.BatchNorm2d(outchannel)  
) if stride != 1 or inchannel != outchannel else  
nn.Identity()
```



ResNet-18

□ 本次实验使用 Resnet-18 作为载体，该网络结构如下所示：

layer name	output size	18-layer	34-layer	50-layer	101-layer	152-layer
conv1	112×112	7×7, 64, stride 2				
		3×3 max pool, stride 2				
conv2_x	56×56	$\begin{bmatrix} 3 \times 3, 64 \\ 3 \times 3, 64 \end{bmatrix} \times 2$	$\begin{bmatrix} 3 \times 3, 64 \\ 3 \times 3, 64 \end{bmatrix} \times 3$	$\begin{bmatrix} 1 \times 1, 64 \\ 3 \times 3, 64 \\ 1 \times 1, 256 \end{bmatrix} \times 3$	$\begin{bmatrix} 1 \times 1, 64 \\ 3 \times 3, 64 \\ 1 \times 1, 256 \end{bmatrix} \times 3$	$\begin{bmatrix} 1 \times 1, 64 \\ 3 \times 3, 64 \\ 1 \times 1, 256 \end{bmatrix} \times 3$
conv3_x	28×28	$\begin{bmatrix} 3 \times 3, 128 \\ 3 \times 3, 128 \end{bmatrix} \times 2$	$\begin{bmatrix} 3 \times 3, 128 \\ 3 \times 3, 128 \end{bmatrix} \times 4$	$\begin{bmatrix} 1 \times 1, 128 \\ 3 \times 3, 128 \\ 1 \times 1, 512 \end{bmatrix} \times 4$	$\begin{bmatrix} 1 \times 1, 128 \\ 3 \times 3, 128 \\ 1 \times 1, 512 \end{bmatrix} \times 4$	$\begin{bmatrix} 1 \times 1, 128 \\ 3 \times 3, 128 \\ 1 \times 1, 512 \end{bmatrix} \times 8$
conv4_x	14×14	$\begin{bmatrix} 3 \times 3, 256 \\ 3 \times 3, 256 \end{bmatrix} \times 2$	$\begin{bmatrix} 3 \times 3, 256 \\ 3 \times 3, 256 \end{bmatrix} \times 6$	$\begin{bmatrix} 1 \times 1, 256 \\ 3 \times 3, 256 \\ 1 \times 1, 1024 \end{bmatrix} \times 6$	$\begin{bmatrix} 1 \times 1, 256 \\ 3 \times 3, 256 \\ 1 \times 1, 1024 \end{bmatrix} \times 23$	$\begin{bmatrix} 1 \times 1, 256 \\ 3 \times 3, 256 \\ 1 \times 1, 1024 \end{bmatrix} \times 36$
conv5_x	7×7	$\begin{bmatrix} 3 \times 3, 512 \\ 3 \times 3, 512 \end{bmatrix} \times 2$	$\begin{bmatrix} 3 \times 3, 512 \\ 3 \times 3, 512 \end{bmatrix} \times 3$	$\begin{bmatrix} 1 \times 1, 512 \\ 3 \times 3, 512 \\ 1 \times 1, 2048 \end{bmatrix} \times 3$	$\begin{bmatrix} 1 \times 1, 512 \\ 3 \times 3, 512 \\ 1 \times 1, 2048 \end{bmatrix} \times 3$	$\begin{bmatrix} 1 \times 1, 512 \\ 3 \times 3, 512 \\ 1 \times 1, 2048 \end{bmatrix} \times 3$
	1×1	average pool, 1000-d fc, softmax				
FLOPs		1.8×10 ⁹	3.6×10 ⁹	3.8×10 ⁹	7.6×10 ⁹	11.3×10 ⁹

ResNet-18

□ ResNet-18网络结构对应pytorch代码如下所示：

```
class ResNet(nn.Module):
    def __init__(self, ResidualBlock, num_classes=10):
        super(ResNet, self).__init__()
        self.inchannel = 64
        self.conv1 = nn.Sequential(
            Conv2d(3, 64, kernel_size=3, stride=1, padding=1, bias=False),
            nn.BatchNorm2d(64),
            nn.ReLU(),
        )
        self.layer1 = self.make_layer(ResidualBlock, 64, 2, stride=1)
        self.layer2 = self.make_layer(ResidualBlock, 128, 2, stride=2)
        self.layer3 = self.make_layer(ResidualBlock, 256, 2, stride=2)
        self.layer4 = self.make_layer(ResidualBlock, 512, 2, stride=2)
        self.fc = nn.Linear(512, num_classes)

    def make_layer(self, block, channels, num_blocks, stride):
        strides = [stride] + [1] * (num_blocks - 1)
        layers = []
        for stride in strides:
            layers.append(block(self.inchannel, channels, stride))
            self.inchannel = channels
        return nn.Sequential(*layers)

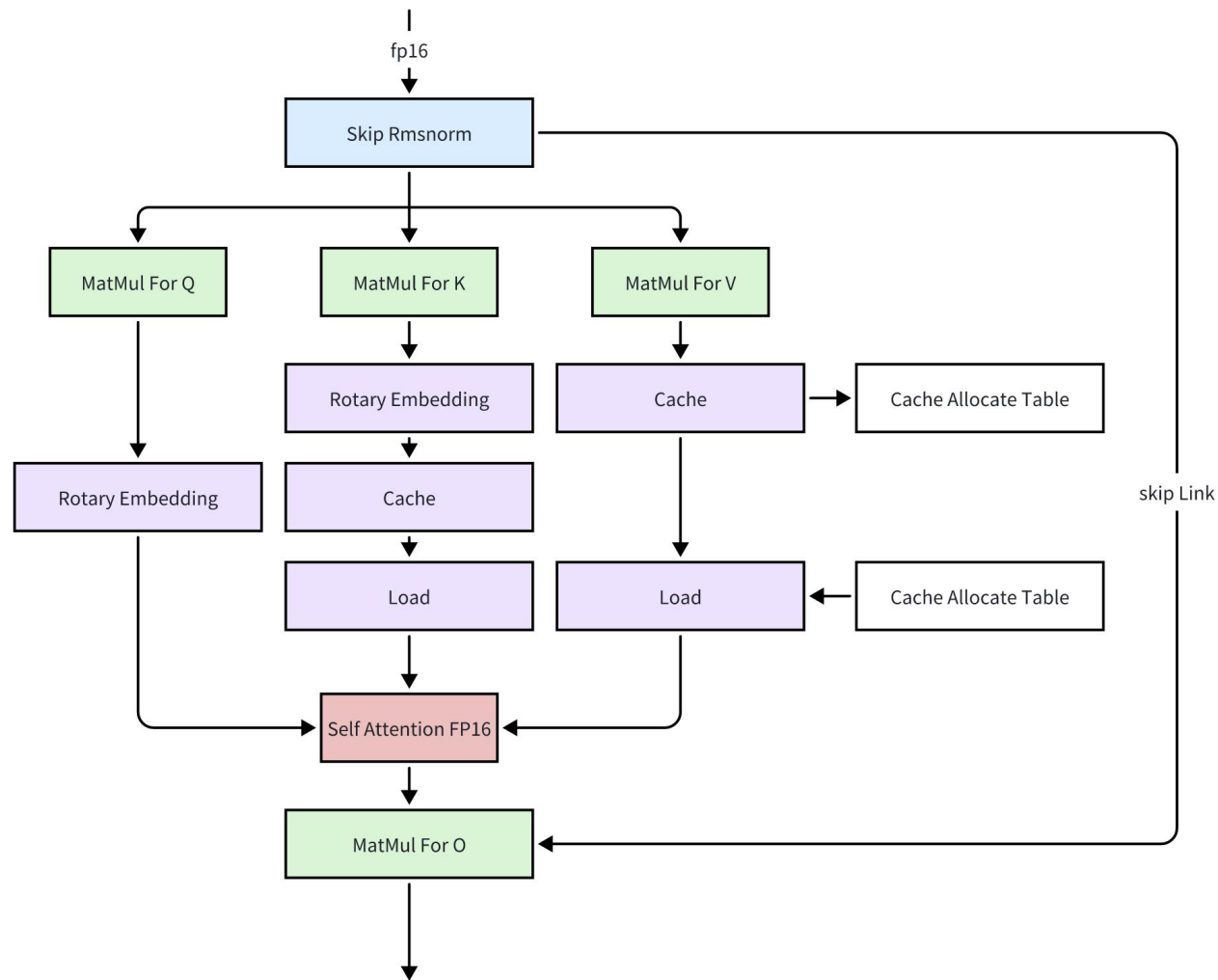
    def forward(self, x):
        out = self.conv1(x)
        out = self.layer1(out)
        out = self.layer2(out)
        out = self.layer3(out)
        out = self.layer4(out)
        out = F.avg_pool2d(out, 4)
        out = out.view(out.size(0), -1)
        out = self.fc(out)
        return out
```

18-layer
$\begin{bmatrix} 3 \times 3, 64 \\ 3 \times 3, 64 \end{bmatrix} \times 2$
$\begin{bmatrix} 3 \times 3, 128 \\ 3 \times 3, 128 \end{bmatrix} \times 2$
$\begin{bmatrix} 3 \times 3, 256 \\ 3 \times 3, 256 \end{bmatrix} \times 2$
$\begin{bmatrix} 3 \times 3, 512 \\ 3 \times 3, 512 \end{bmatrix} \times 2$
1.8×10^9

背景知识：模型量化

- 近年来，随着Transformer、MOE架构的提出，使得深度学习模型轻松突破上千亿、万亿规模参数，从而导致模型变得越来越大。

Transformer



背景知识：模型量化

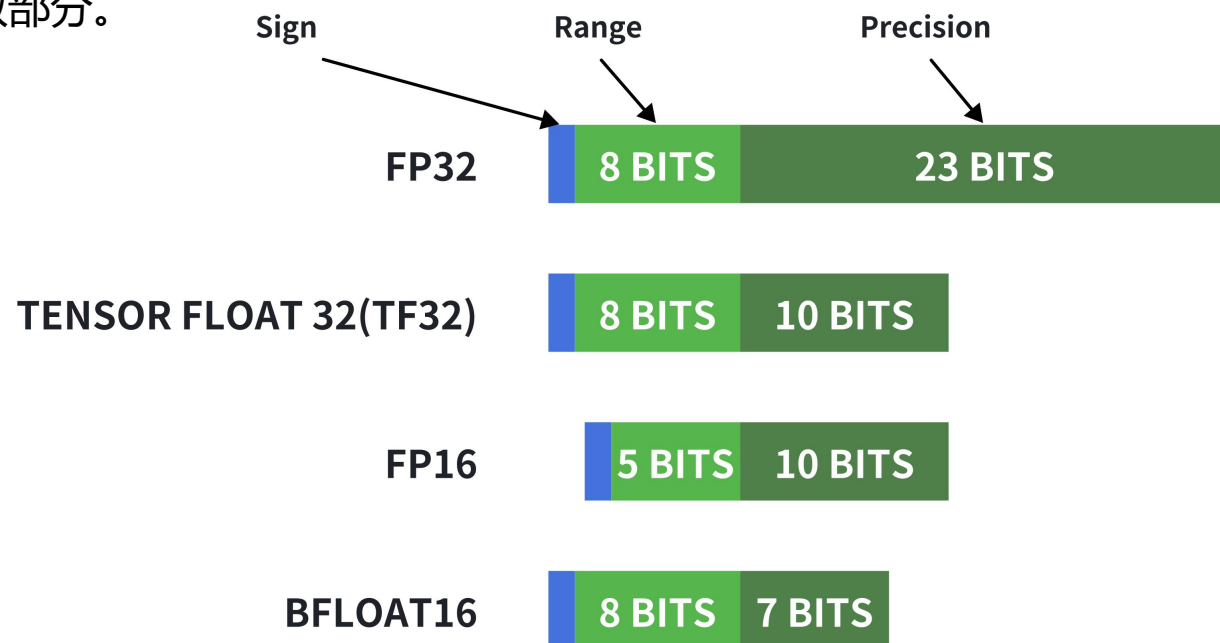
- 因此，我们需要一些模型压缩技术来降低模型部署的成本，并提升模型的推理性能。模型压缩技术主要分为如下几类：
 - 剪枝：剪枝是一种通过减少模型中的参数量来进行模型压缩的技术，可以在保证一定精度的情况下减少模型的内存占用和硬件消耗。
 - 知识蒸馏：知识蒸馏是一种技术，可以将知识从计算成本较高的大型模型转移到较小的模型，而不会失去有效性。
 - 量化（Quantization）：模型量化是一种压缩参数的方式，它将神经网络的权重、特征图等原本用浮点表示的量值换用定点（整型）表示，模型量化是大模型压缩部署最重要的技术。
- 本次实验我们会通过**量化技术**实现模型推理加速。

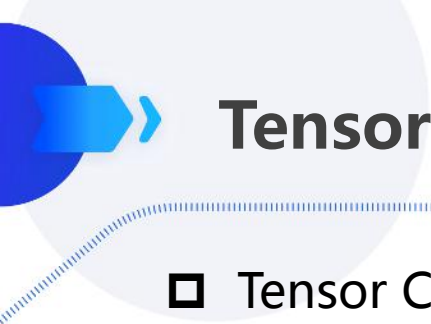
量化的对象

- 在深度学习中，量化对象指的是可被转换为低位宽格式的关键数据单元，以下四类对象通常被量化：
 - 权重：模型权重是静态参数，量化权重可达到减少模型大小内存和占用空间。
 - 激活值：激活值是网络层输出的动态中间结果，占据推理时大部分内存。量化激活值可显著减少内存占用，也可与量化后的权重结合，加速整数计算。
 - KV cache：在生成任务中，KV Cache 存储历史序列的键值对，其内存随序列长度线性增长。量化 KV Cache 可将长文本生成的吞吐量提升 2-3 倍，解决显存瓶颈问题。
 - 梯度：相对上面三者略微小众一些，因为主要用于训练。在训练深度学习模型时，梯度通常是浮点数，它主要作用是在分布式计算中减少通信开销，同时，也可以减少反向传播的开销。

量化的格式

- 数值量化旨在将高精度浮点表示（如FP32）映射到低位宽格式（如FP16、Int8等），以降低计算开销、内存占用及功耗，同时尽可能保留模型性能。主流的量化格式有下面这些：
- Float32 (FP32)：标准的 IEEE 32 位浮点表示，指数 8 位，尾数 23 位，符号 1 位，大部分硬件都支持 FP32 运算指令。
- Float16 (FP16)：指数 5 位，尾数 10 位，符号 1 位。FP16 数字的数值范围远低于 FP32，存在上溢和下溢 的风险。
- Bfloat16 (BF16)：指数 8 位 (与 FP32 相同)，尾数 7 位，符号 1 位。这意味着 BF16 可以保留与 FP32 相同的动态范围。但是相对于 FP16，损失了 3 位精度。因此，在使用 BF16 精度时，大数值绝对没有问题，但是精度会比 FP16 差。
- Int8：使用 8 位表示，一位符号位7位数值，没有小数部分。





Tensor core介绍

- Tensor Core 是 NVIDIA GPU 中专用于加速矩阵乘累加 (Matrix Multiply-Accumulate, MMA) 运算的专用计算单元，每个 Tensor Core 可单周期执行 4x4x4 (FP16) 或 8x8x4 (Int8) 的矩阵块乘累加运算，**计算密度远超传统 CUDA Core。**
- 以 NVIDIA A100 PCIE版为例，该卡的第三代 Tensor Core 技术可为各种负载提供加速支持。

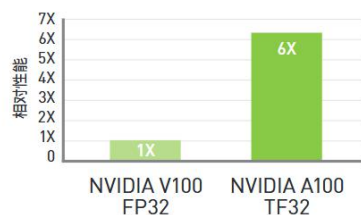
计算类型	算力 (TFLOPS)	计算密度 (vs CUDA Core)	适用场景
FP32 (CUDA)	19.5	1x	通用计算
FP16 (Tensor)	624	32x	混合精度训练
TF32 (Tensor)	312	16x	替代 FP32 训练
Int8 (Tensor)	1248 (TOPS)	64x	推理加速
INT4(Tensor)	2496 (TOPS)	128x	推理加速

在各个规模上实现出色加速

NVIDIA A100 Tensor Core GPU 可针对 AI、数据分析和高性能计算 (HPC) 在各个规模上实现出色加速，从而应对极其严峻的计算挑战。作为 NVIDIA 数据中心平台的引擎，A100 既可高效扩展至数千个 GPU，也可通过全新多实例 GPU (MIG) 技术分割为七个独立的 GPU 实例，从而加速各个规模的工作负载。A100 的第三代 Tensor Core 技术现可为各种工作负载的更多精度水平提供加速支持，缩短获取洞见以及产品上市所需的时间。

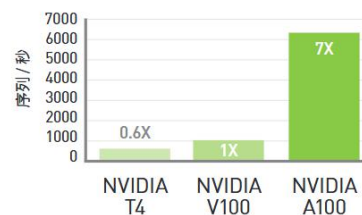
借助 TF32 精度, 开箱即用的 AI 训练性能至多可提高至 6 倍¹

BERT Large 训练

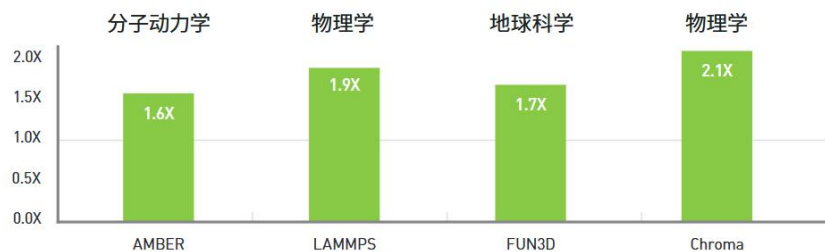


借助多实例 GPU (MIG), AI 推理性能至多可提高至 7 倍²

BERT Large 推理



至多可将 HPC 性能提高至 2 倍³



系统规格 (峰值性能)

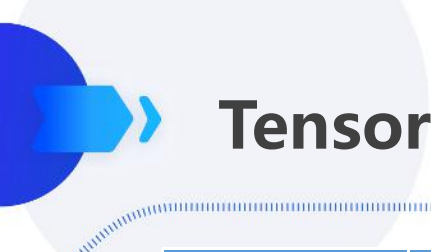
	NVIDIA HGX™ 版 NVIDIA A100	PCIe 版 NVIDIA A100
GPU 架构	NVIDIA Ampere	
双精度浮点运算性能	FP64 : 9.7 TFLOPS FP64 Tensor Core : 19.5 TFLOPS	
单精度浮点运算性能	FP32 : 19.5 TFLOPS Tensor Float 32 (TF32) : 156 TFLOPS 312 TFLOPS*	
半精度浮点运算性能	312 TFLOPS 624 TFLOPS*	
Bfloat16 浮点运算性能	312 TFLOPS 624 TFLOPS*	
整数运算性能	INT8 : 624 TOPS 1248 TOPS* INT4 : 1248 TOPS 2496 TOPS*	
GPU 显存	40 GB HBM2	
显存带宽	1.6 TB/s	
纠错码	支持	
互联接口	PCIe 4.0 : 64 GB/s 第三代 NVIDIA® NVLink® : 600 GB/s**	PCIe 4.0 : 64 GB/s 第三代 NVIDIA® NVLink® : 600 GB/s**
结构外形	NVIDIA HGX™ A100 中 可选 4 或 8 个 SXM GPU	PCIe
多实例 GPU (MIG)	高达 7 个 GPU 实例	
最大功耗	400 瓦	250 瓦
提供顶级应用性能	100%	90%
散热解决方案	被动式	
计算 API	CUDA®、DirectCompute、 OpenCL™、OpenACC®	

* 启用结构化稀疏技术

** SXM GPU 通过 HGX A100 服务器主板连接, PCIe GPU 通过 NVLink 桥接器可桥接多达 2 块 GPU。

Tensor core架构发展

- ❑ 2017 年，Volta 架构横空出世，其中**引入张量核心**（Tensor Core）设计可谓划时代之作，这一设计专门针对深度学习计算进行了优化，通过执行融合乘法加法操作，大幅提升了计算效率。
- ❑ 紧随其后，在一年后的 2018 年，英伟达发布了 Turing 架构，进一步增强了 Tensor Core 的功能。Turing 架构不仅延续了对浮点运算的优化，还新增了对 **INT8、INT4**、甚至是 Binary(INT1)等整数格式的支持。
- ❑ 2020 年，Ampere 架构的推出再次刷新了人们对 Tensor Core 的认知。Ampere 架构新增对 **TF32 和 BF16** 两种数据格式的支持，这些新的数据格式进一步提高了深度学习训练和推理的效率。同时，Ampere 架构引入了对稀疏矩阵计算的支持。
- ❑ 到了 2022 年，英伟达发布了专为深度学习设计的 Hopper 架构。Hopper 架构标志性的变化是引入了 **FP8 张量核心**，这一创新进一步加速了 AI 训练和推理过程。
- ❑ 2024 年，英伟达推出了 Blackwell 架构为生成式 AI 带来了显著的飞跃。相较于 H100 GPU，GB200 Superchip 在处理 LLM 推理任务时，性能实现了高达 30 倍的惊人提升，同时在能耗方面也实现了高达 25 倍的优化。



Tensor core架构发展

架构名称	Volta	Turing	Ampere	Hopper
中文名字	伏特	图灵	安培	赫柏
发布时间	2017	2018	2020	2022
核心参数	80个SM，每个SM包括32个FP64+64 Int32+64 FP32+8个Tensor Cores	102核心92个SM，SM重新设计，每个SM包含64个Int32+64个FP32+8个Tensor Cores	108个SM，每个SM包含64个FP32+64个INT32+32个FP64+4个Tensor Cores	132个SM，每个SM包含128个FP32+64个INT32+64个FP64+4个Tensor Cores
特点&优势	NVLink2.0，Tensor Cores第一代，支持AI运算	Tensor Core2.0，RT Core第一代	Tensor Core3.0，RT Core2.0，NVLink3.0，结构稀疏性矩阵MIG1.0	Tensor Core4.0，NVlink4.0，结构稀疏性矩阵MIG2.0
纳米制程	12nm211亿晶体管	12nm186亿晶体管	7nm283亿晶体管	4nm800亿晶体管
代表型号	V100 TiTan V	T4，2080TI RTX 5000	A100 A30系列	H100



Tensor core API介绍

- Tensor Core 通过 Warp Matrix Multiply-Accumulate (**WMMA**) API 编程，核心接口如下：

```
// 定义矩阵片段 (Fragment)
template <typename Use, int M, int N, int K, typename T, typename Layout>
class fragment;

// 加载矩阵到片段
void load_matrix_sync(fragment<...> &a, const T* ptr, int ldm);
void load_matrix_sync(fragment<...> &a, const T* ptr, int ldm, layout_t layout);

// 执行矩阵乘累加
void mma_sync(fragment<...> &d, const fragment<...> &a,
              const fragment<...> &b, const fragment<...> &c);

// 存储片段到内存
void store_matrix_sync(T* ptr, const fragment<...> &a, int ldm, layout_t layout);
```

Tensor core API介绍

□ 使用 wmma 实现FP16矩阵乘法的程序：

```
#include <cuda_fp16.h>
#include <cuda_runtime.h>
#include <mma.h>
using namespace nvcuda;

__global__ void wmma_gemm(half *a, half *b, float *c, int M, int N, int K) {
    // 声明矩阵片段 (Fragment)
    wmma::fragment<wmma::matrix_a, 16, 16, 16, half, wmma::row_major> a_frag;
    wmma::fragment<wmma::matrix_b, 16, 16, 16, half, wmma::col_major> b_frag;
    wmma::fragment<wmma::accumulator, 16, 16, 16, float> c_frag;

    // 初始化累加器
    wmma::fill_fragment(c_frag, 0.0f);

    for (int ki = 0; ki < K; ki += 16) {
        int a_row = blockIdx.y * 16;
        int b_col = blockIdx.x * 16;
        // 加载矩阵到片段
        wmma::load_matrix_sync(a_frag, a + a_row * K + ki, K);
        wmma::load_matrix_sync(b_frag, b + ki * N + b_col, N);
        // 执行矩阵乘累加
        wmma::mma_sync(c_frag, a_frag, b_frag, c_frag);
    }
    // 存储片段到显存
    wmma::store_matrix_sync(c + blockIdx.y * 16 * N + blockIdx.x * 16, c_frag, N, wmma::mem_row_major);
}

.....
```



Tensor core API介绍

□ 声明矩阵片段(fragment)：

代码如下：

```
wmma::fragment<wmma::matrix_a, 16, 16, 16, half, wmma::row_major> a_frag;  
wmma::fragment<wmma::matrix_b, 16, 16, 16, half, wmma::col_major> b_frag;  
wmma::fragment<wmma::accumulator, 16, 16, 16, float> c_frag;
```

WMMA api主要用于实现 GEMM 运算，因此需要指定片段为 输入矩阵A或B或累加器。

在这里我们指定块大小为 $M=16, N=16, K=16$, 表示Tensor core一次可以处理 $M=16, N=16, K=16$ 的GEMM。

我们继续指定输入矩阵为FP16, 累加器为FP32，同时A按行主序存储(row_major), B按列存储(col_major)

□ 初始化累加器：

矩阵乘法需要将累加器片段 c_frag 初始化为0，后续通过累加操作逐步计算结果。

```
wmma::fill_fragment(c_frag, 0.0f);
```

Tensor core API介绍

□ 分块加载与计算与写回：

wmma API全名为：Warp-level Matrix Multiply Accumulate，这表明计算和访存以warp为基本单位。

下述程序表明：每个warp（warp内32个线程）共同加载一个 16x16 的A块数据和一个 16x16 的B块数据，计算结果 c_frag 由整个Warp共享，最终得到 16x16 的输出块。

```
for (int ki = 0; ki < K; ki += 16) {  
    // 加载 A 的 16x16 块（行主序）  
    wmma::load_matrix_sync(a_frag, a + threadIdx.y * 16 * K + ki, K);  
    // 加载 B 的 16x16 块（列主序）  
    wmma::load_matrix_sync(b_frag, b + ki * N + threadIdx.z * 16, N);  
    // 执行矩阵乘累加: c_frag += a_frag * b_frag  
    wmma::mma_sync(c_frag, a_frag, b_frag, c_frag);  
}  
wmma::store_matrix_sync(c + blockIdx.y * 16 * N + blockIdx.x * 16, c_frag, N, wmma::mem_row_major);
```


实验项目介绍

- Lab6 旨在探索卷积算子的全流程优化策略，具体而言需要同学们：
 - 1. 使用实验三的隐式卷积代码（FP32）替换Pytorch框架的conv2d方法，执行inference.py验证推理精度与吞吐量。
 - 2. 将实验三的隐式卷积代码迁移到FP16，再次替换Pytorch框架的conv2d方法，执行inference.py验证推理精度与吞吐量
- 本次实验配套了项目Lab6_experiment，该项目整体架构如下所示：

```
JNU_lab_experiment4/  
├── build/  
├── cpp/  
│   └── conv2d_xxx.cpp  
├── cuda/  
│   └── conv2d_xxx_kernel.cu  
├── include/  
│   ├── conv2d_fp16.h  
│   └── conv2d_fp32.h  
├── modules/  
│   ├── conv_layer_xx.py  
│   └── resnet_18_xx.py  
├── pytorch/  
│   ├── data/  
│   └── model/  
├── inference.py  
└── setup.sh
```

cifa数据集
resnet-18模型
推理程序
算子注册脚本

算子注册

□ 为了替换Pytorch的conv2d方法，我们划分出下面三层架构：

层级	组件文件	技术职责
注册层	setup.py	联合编译 C++/CUDA 代码为 Python 可调用的库
调度层	conv2d_optim_fp16.cpp	张量合法性校验、内存分配、核函数参数封装、CUDA流控制
实现层	conv2d_optim_kernel_xx.cu	实现 xx 精度的 Implicit GEMM 算法，包含前向/反向计算的优化逻辑

算子注册-注册层

- 注册层 setup.py 定义了三个库：conv2d_optim_fp16, conv2d_optim_fp32 和 conv2d_baseline_fp32，算子注册成功后python程序即可调用这些库。

```
from setuptools import setup, find_packages
from torch.utils.cpp_extension import BuildExtension, CUDAExtension
```

```
# 基于 implicitGemm fp16 的卷积实现
```

```
setup(
    name='conv2d_optim_fp16',
    include_dirs=['include'],
    ext_modules=[
        CUDAExtension('conv2d_optim_fp16', [
            'cpp/conv2d_optim_fp16.cpp',
            'cuda/conv2d_optim_kernel_fp16.cu',
        ]),
    ],
    cmdclass={
        'build_ext': BuildExtension
    })
```

```
# 基于 implicitGemm fp32 的卷积实现
```

```
setup(
    name='conv2d_optim_fp32',
    include_dirs=['include'],
    ext_modules=[
        CUDAExtension('conv2d_optim_fp32', [
            'cpp/conv2d_optim_fp32.cpp',
            'cuda/conv2d_optim_kernel_fp32.cu',
        ]),
    ],
    cmdclass={
        'build_ext': BuildExtension
    })
```

```
# 基于 直接卷积 的卷积实现
```

```
setup(
    name='conv2d_baseline_fp32',
    include_dirs=['include'],
    ext_modules=[
        CUDAExtension('conv2d_baseline_fp32', [
            'cpp/conv2d_baseline_fp32.cpp',
            'cuda/conv2d_baseline_kernel_fp32.cu',
        ]),
    ],
    cmdclass={
        'build_ext': BuildExtension
    })
```

```
import torch
import torch.nn as nn
import conv2d_optim_fp16 as conv2d
import math
from torch.nn.common_types import _size_1_t, _size_2_t, _size_3_t
from torch.nn.modules.utils import _single, _pair, _triple, _reverse_repeat_tuple

from typing import Optional, List, Tuple, Union

def nchw_to_nhwc(x):
    return x.permute(0,2,3,1).contiguous()

def nhwc_to_nchw(x):
    return x.permute(0,3,1,2).contiguous()

class Conv2DFunction(torch.autograd.Function):
    @staticmethod
    def forward(ctx, input, weight, params):
        stride = _pair(params[0])
        padding = _pair(params[1])
        #input_nhwc = nchw_to_nhwc(input.contiguous())
        output_nhwc = conv2d.forward(input, weight, stride, padding)
        #output = nhwc_to_nchw(output_nhwc.contiguous())
        return output_nhwc

    @staticmethod
    def backward(ctx, grad_output):
        input, weight, params = ctx.saved_variables
        stride = _pair(params[0])
        padding = _pair(params[1])
        grad_input, grad_weight = conv2d.backward(input.contiguous(), grad_output.contiguous(), weight.contiguous())
        return grad_input, grad_weight, None
```

算子注册-调度层

- setup.py 依赖cpp文件和cu文件，cpp文件作为调度层，功能类似胶水，用于连接CUDA具体实现与Python接口定义。如下图所示，cpp文件定义了conv2d_forward方法，通过调用CUDA conv2d_cuda_forward() 实现具体计算。

```
torch::Tensor conv2d_forward(
    torch::Tensor input,
    torch::Tensor weight,
    torch::IntArrayRef stride,
    torch::IntArrayRef padding)
{
    CHECK_INPUT(input);
    CHECK_INPUT(weight);

    // Convolution parameter

    param_t param;
    param.input = (DTYPE *)input.data_ptr();
    param.weight = (DTYPE *)weight.data_ptr();
    param.n = input.size(0);
    param.c = input.size(1);
    param.h = input.size(2);
    param.w = input.size(3);
    param.k = weight.size(0);
    param.r = weight.size(2);
    param.s = weight.size(3);
    param.u = stride[0];
    param.v = stride[1];
    param.p = padding[0];
    param.q = padding[1];

    int64_t outh = (param.h - param.r + 2 * param.p) / param.u + 1;
    int64_t outw = (param.w - param.s + 2 * param.q) / param.v + 1;

    param.Oh = outh;
    param.Ow = outw;

    auto output = torch::zeros(torch::IntArrayRef({input.size(0), weight.size(0), outh, outw}),
                                input.options());

    param.output = (DTYPE *)output.data_ptr();
    conv2d_cuda_forward(param);
    return output;
}
```

算子注册-调度层

- 有了具体实现下一步就是将其与Python接口绑定。
- 具体而言cpp文件使用 pybind11 将CUDA 实现的卷积前向/反向函数暴露给 Python, 使其可在PyTorch中调用。

```
// Python module interface bind
PYBIND11_MODULE(TORCH_EXTENSION_NAME, m)
{
    m.def("forward", &conv2d_forward, "Convolution2d forward (CUDA)");
    m.def("backward", &conv2d_backward, "Convolution2d backward (CUDA)");
}
```

算子注册-实现层

- cuda/conv2d_kernel_xx.cu 负责算子实现，如图所示 conv_cuda_forward 配置了线程信息并执行具体的核函数：

```
void conv2d_cuda_forward(param_t param) {  
    // 调整grid.y以适应4倍k通道 (原block.y从16变为64)  
    int blockx = (param.Oh * param.Ow + 15) / 16;  
    int blocky = (param.k + 63) / 64; // 每个block处理64个k  
    int blockz = param.n;  
    dim3 block(16, 16, 1); // thread.y=16, 每个线程处理4个k  
    dim3 grid(blockx, blocky, blockz);  
    implgemm<<<grid, block>>>(param);  
}
```


你需要做什么

□ 从实验角度看你需要做以下工作：

- 将cuda/conv2d_optim_kernel_fp32.cu 并做适当调整以符合cpp接口要求。
 - 将cuda/conv2d_optim_kernel_fp32.cu CUDA 实现移植到 cuda/conv2d_optim_kernel_fp16.cu，由于fp16的数据类型__half不支持重载，因此需要同学参考下述网站对运算符做替换：https://docs.nvidia.com/cuda/cuda-math-api/cuda_math_api/group__CUDA__MATH__HALF__ARITHMETIC.html
 - 根据任务在inference.py 选择合适的数据类型(FP16 / FP32)，并执行python inference.py验证推理精度和吞吐量。
- fp32的赋初值：

□ fp32的赋初值方法：

```
else
{
    gradoutput_ldg_reg[i] = 0.0f;
}
```

□ fp16的赋初值方法：

```
} else {
    sh_input[ty][tx] = __float2half(0.0f);
}
```

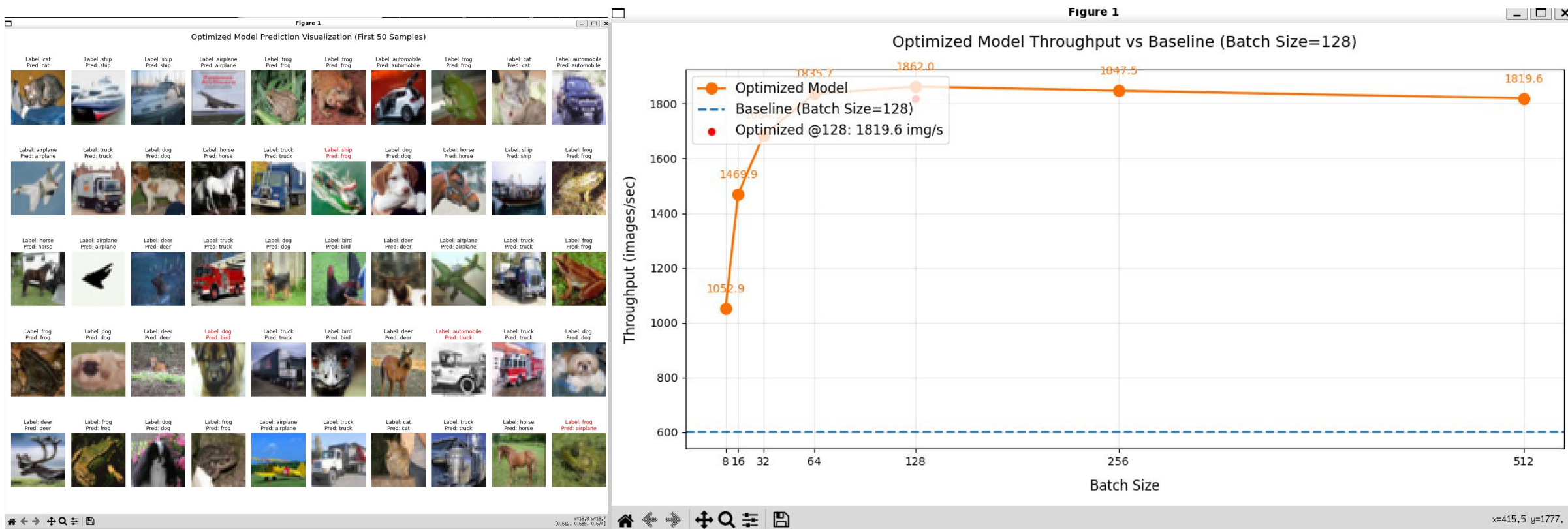
inference.py 程序中选择合适的数据类型做推理：

```
MODEL_PATH = "./pytorch/model/net_123.pth"
DATA_ROOT = "./pytorch/data"
MODEL_DTYPE = "FP16"
BATCH_SIZES = [8, 16, 32, 64, 128, 256, 512]
NUM_WORKERS = 4
RUN_TIMES = 1
DEVICE = torch.device("cuda" if torch.cuda.is_available() else "cpu")

if MODEL_DTYPE == "FP16":
    from modules.resnet_18_optim_fp16 import ResNet18 as ResNet18_optim
elif MODEL_DTYPE == "FP32":
    from modules.resnet_18_optim_fp32 import ResNet18 as ResNet18_optim
else:
    from modules.resnet_18_baseline_fp32 import ResNet18 as ResNet18_optim
```


你需要做什么

- 本次提供的模型精度在 90% 以上，如果推理精度低于该指标则说明你的算子实现有误。
- 执行python inference.py 后你会看到前50个样本的标签对比以及不同batchSize下的推理吞吐量。
- 我们的目标是：在模型精度不变的情况下尽可能提高推理吞吐量。



实验要求说明

□ 基本要求（必做）

- 请同学们基于上述教程完成 `conv2d_optim_kernel_fp32.cu` 、 `conv2d_optim_kernel_fp16.cu` 相关代码，注册算子并推理模型。**你的代码需要满足以下约束条件：**

- 1. 推理精度 $\geq 90\%$
- 2. 推理吞吐量较baseline有提高

□ 进阶要求（选做）

- 完成程序 `test_wmma.cu` ，比较Tensor core与Cuda core的性能差距。
 - 编译命令：`nvcc -arch=sm_89 -o test test_wmma.cu`
- 使用 wmma API，基于 im2col+GEMM 或 implicitGEMM 优化模型推理，约束条件与基本要求一致。
- 发挥想象，在精度不变前提下提高推理吞吐量。